

题目链接

本题是2011年ICPC亚洲区域赛大连赛区的F题

题意

N*M的矩阵，总共个有9种颜色，有4种操作，每种操作在一定范围内涂上某种颜色，后涂的可以覆盖前面涂的，问最后每种颜色有多少个。

分析

首先线段树可以做，并且行数很少，不需要用二维线段树，按行数开线段树数组即可。

但其实用并查集做更优，并查集在处理一些区间问题（区间合并快速跳表，区间覆盖路径压缩）上有奇效。

并查集处理思路：对每行进行并查集路径压缩。倒着处理，相当于先涂上的颜色不会被后涂上的颜色覆盖，pa为下一个未涂色的结点，那么对行处理路径压缩就会跳过已经涂色的结点，比如当前行第i列涂完了下次就要涂pa[i+1]，并且把pa[i+1]也赋给pa[i]。

本题的坑点：1、坐标越界检查；2、对于画三角形，题目虽然说w是奇数，但测试数据有出现w是偶数的情况，将偶数-1变成奇数才AC。

AC代码

线段树版本

```
#include <iostream>
#include <cmath>
using namespace std;

#define N 50050
int e[1<<17], g[N], x[N], y[N], r[N], w[N], c[N], l[N], a[10], m, n, q, b, t, cc; char s[10];

void set(int o, int l, int r) {
    if (l >= b && r <= t) {
        e[o] = cc; return;
    }
    int m = (l+r) >> 1, lc = o << 1, rc = lc + 1;
    if (e[o] >= 0) e[lc] = e[rc] = e[o], e[o] = -1;
    if (b <= m) set(lc, l, m);
    if (t > m) set(rc, m+1, r);
}

void query(int o, int l, int r) {
    if (e[o] >= 0) {
        a[e[o]] += min(r, t) - max(l, b) + 1;
        return;
    }
    int m = (l+r) >> 1, lc = o << 1, rc = lc + 1;
    if (b <= m) query(lc, l, m);
    if (t > m) query(rc, m+1, r);
}

void solve() {
    for (int i=1; i<10; ++i) a[i] = 0;
    for (int i=0; i<q; ++i) {
        cin >> s >> x[i] >> y[i]; g[i] = s[0];
        s[0] == 'R' ? cin >> l[i] >> w[i] >> c[i] : (s[0] == 'T' ? cin >> w[i] >> c[i] : cin >> r[i] >> c[i]);
    }
    for (int i=0; i<n; ++i) {
        e[1] = 0;
        for (int j=0; j<q; ++j) {
            if (g[j] == 'C') {
                if (i < x[j]-r[j] || i > x[j]+r[j]) continue;
                int h = sqrt(r[j]*r[j] - (i-x[j])*(i-x[j]) + .5);
                b = max(y[j]-h, 0); t = min(y[j]+h, m-1);
            } else if (g[j] == 'D') {
                if (i < x[j]-r[j] || i > x[j]+r[j]) continue;
                b = max(y[j]-r[j]+abs(i-x[j]), 0); t = min(y[j]+r[j]-abs(i-x[j]), m-1);
            } else if (g[j] == 'R') {
                if (i < x[j] || i >= x[j]+l[j]) continue;
                b = y[j]; t = min(y[j]+w[j], m)-1;
            } else {
                int h = (w[j]-1) >> 1;
                if (i < x[j] || i > x[j]+h) continue;
                b = max(y[j]-h+i-x[j], 0); t = min(y[j]+h-i+x[j], m-1);
            }
            cc = c[j]; set(1, 0, m-1);
        }
        b = 0; t = m-1; query(1, 0, m-1);
    }
    cout << a[1]; for (int i=2; i<10; ++i) cout << ' ' << a[i]; cout << endl;
}

int main() {
    ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
    while (cin >> n >> m >> q) solve();
    return 0;
}
```

并查集版本

```
#include <iostream>
#include <cmath>
using namespace std;

#define N 50050
int f[202][N], g[N], x[N], y[N], r[N], w[N], c[N], l[N], a[10], m, n, q; char s[10];

int find(int (&f)[N], int x) {
    return x == f[x] ? x : f[x] = find(f, f[x]);
}

void solve() {
    for (int i=1; i<10; ++i) a[i] = 0;
    for (int i=0; i<q; ++i) {
        cin >> s >> x[i] >> y[i]; g[i] = s[0];
        s[0] == 'R' ? cin >> l[i] >> w[i] >> c[i] : (s[0] == 'T' ? cin >> w[i] >> c[i] : cin >> r[i] >> c[i]);
    }
    for (int i=0; i<n; ++i) for (int j=0; j<=m; ++j) f[i][j] = j;
    while (q--) {
        if (g[q] == 'C') {
            for (int i=max(x[q]-r[q], 0), j=min(x[q]+r[q], n-1); i<=j; ++i) {
                int h = sqrt(r[q]*r[q] - (i-x[q])*(i-x[q]) + .5), b = find(f[i], max(y[q]-h, 0)), t = min(y[q]+h, m-1);
                while (b <= t) ++ a[c[q]], b = f[i][b] = find(f[i], b+1);
            }
        } else if (g[q] == 'D') {
            for (int i=max(x[q]-r[q], 0), j=min(x[q]+r[q], n-1); i<=j; ++i) {
                int b = find(f[i], max(y[q]-r[q]+abs(i-x[q]), 0)), t = min(y[q]+r[q]-abs(i-x[q]), m-1);
                while (b <= t) ++ a[c[q]], b = f[i][b] = find(f[i], b+1);
            }
        } else if (g[q] == 'R') {
            for (int i=x[q], j=min(x[q]+l[q], n); i<=j; ++i) {
                int b = find(f[i], y[q]), t = min(y[q]+w[q], m)-1;
                while (b <= t) ++ a[c[q]], b = f[i][b] = find(f[i], b+1);
            }
        } else {
            for (int i=x[q], h=(w[q]-1)>>1, j=min(x[q]+h, n-1); i<=j; ++i) {
                int b = find(f[i], max(y[q]-h+i-x[q], 0)), t = min(y[q]+h-i+x[q], m-1);
                while (b <= t) ++ a[c[q]], b = f[i][b] = find(f[i], b+1);
            }
        }
    }
    cout << a[1]; for (int i=2; i<10; ++i) cout << ' ' << a[i]; cout << endl;
}

int main() {
    ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
    while (cin >> n >> m >> q) solve();
    return 0;
}
```